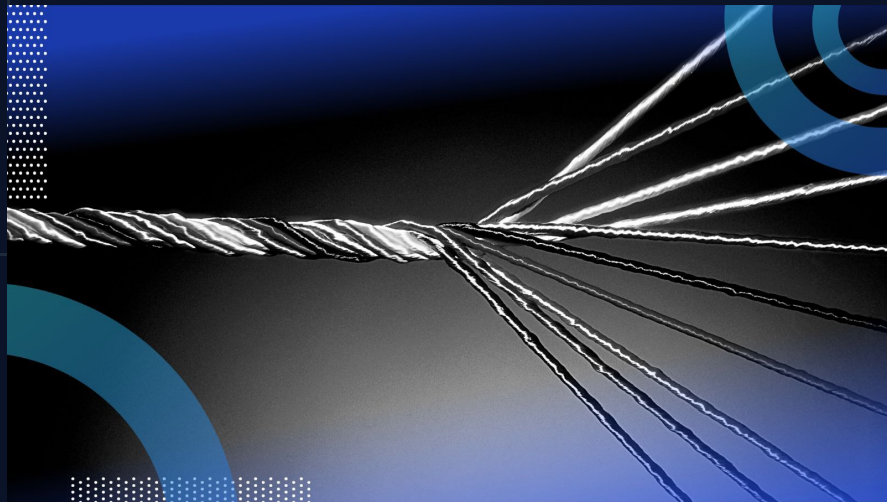


Scheduling & Multithreading

Principles of CPU Scheduling & Concurrent Execution

- Efficient hardware resource management
- Sequential execution to concurrency & parallelism
- Architectural focus, language & OS independent



Processes vs. Threads

Execution Units & Resource Sharing

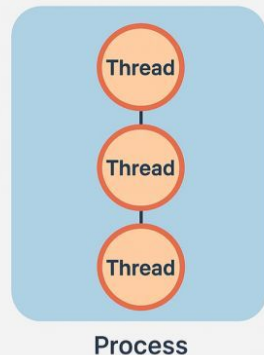
Process

An autonomous unit of execution characterized by its own isolated address space, which completely encapsulates its own dedicated code, data segments, and system resources.

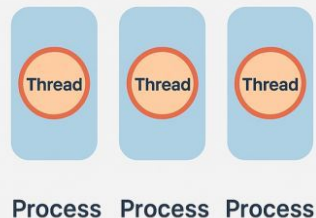
Thread LWP

A lightweight sub-unit of execution operating within the boundaries of a parent process. Multiple threads run concurrently to maximize CPU utilization.

MULTI-THREADING



MULTI-PROCESSING



- **Resource Sharing:** Threads share global memory & process resources, but maintain an independent stack, registers, and Program Counter (PC).
- **The Advantage:** Eliminating isolated address boundaries allows thread context switching to perform significantly faster than process switching.

States of Execution Units

Task Lifecycle and States

Lifecycle States

- **Running:** The task is actively occupying the CPU and executing instructions.
- **Ready:** The task is prepared for execution, waiting in the run queue for the scheduler's decision.
- **Blocked / Waiting:** The task is suspended, waiting for an external event (I/O, resource allocation, or synchronization).

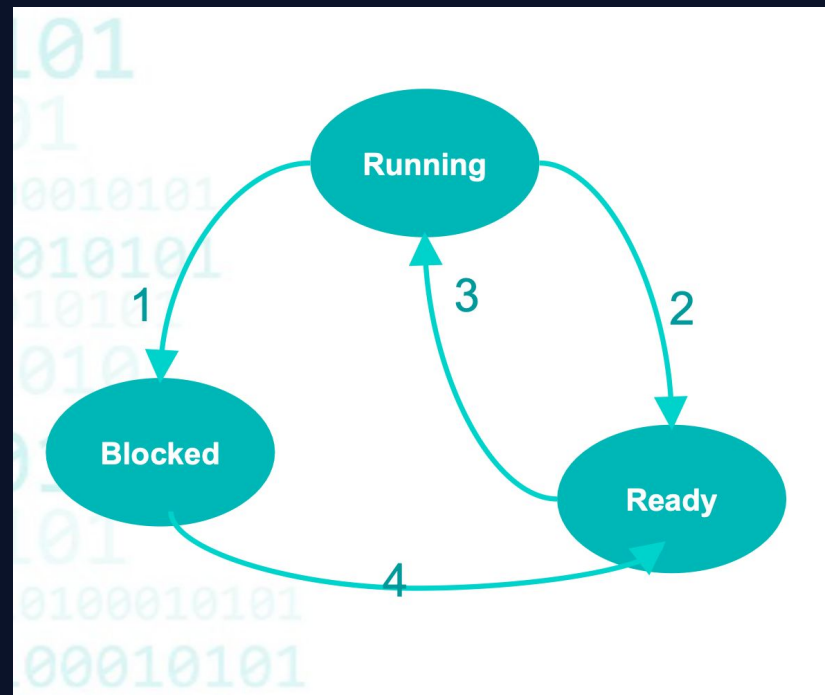
State Transitions

Running → Blocked: I/O or event request

Blocked → Ready: Event completion

Running → Ready: Preemption (time slice end)

Ready → Running: CPU dispatch



What is Scheduling & Why Do We Need It?

Fundamentals of Scheduling & Resource Allocation

Definition

The operating system algorithm designed specifically to decide and control the distribution of CPU time among executing tasks, ensuring resource allocation.

Core Objectives

- **Maximize CPU Utilization:** Keep the processor actively executing tasks to avoid wasted idle cycles.
- **Minimize Response Time:** Ensure highly interactive systems react rapidly to user requests.
- **Ensure Fairness:** Distribute processor access equitably across all executing tasks.

Execution Policies

Non-Preemptive Scheduling

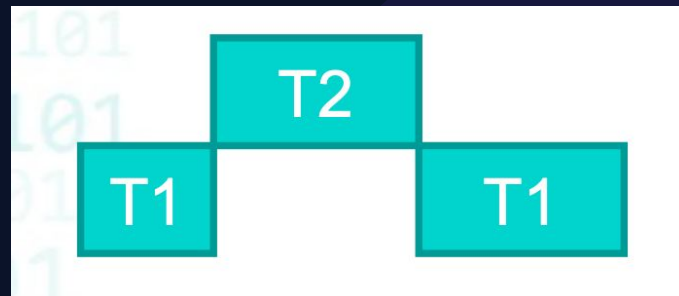
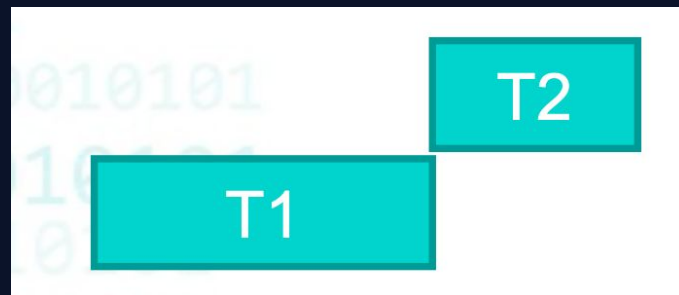
Voluntary Yield

A running task completely retains control of the CPU, voluntarily yielding execution only upon completion or when blocked by an external event.

Preemptive Scheduling

Forced Interruption

The OS retains overall authority and can forcibly interrupt a running task to allocate processor time to another thread (e.g., during time-slice expiration).



Linux Scheduling Algorithms & Policies

Classical & Real-Time Scheduling Algorithms in Linux

STATIC PRIORITY 0

General-Purpose Policies

SCHED_OTHER (SCHED_NORMAL)

Standard Linux time-sharing policy using dynamic priorities adjusted via nice levels (-20 to +19).

SCHED_BATCH

Optimized for CPU-intensive background tasks to minimize context-switching overhead.

SCHED_IDLE

Runs tasks only when the CPU has no other work.

STATIC PRIORITY 1 TO 99

Real-Time Policies

SCHED_FIFO

Preemptive First-In, First-Out real-time policy; runs until blocked or yields.

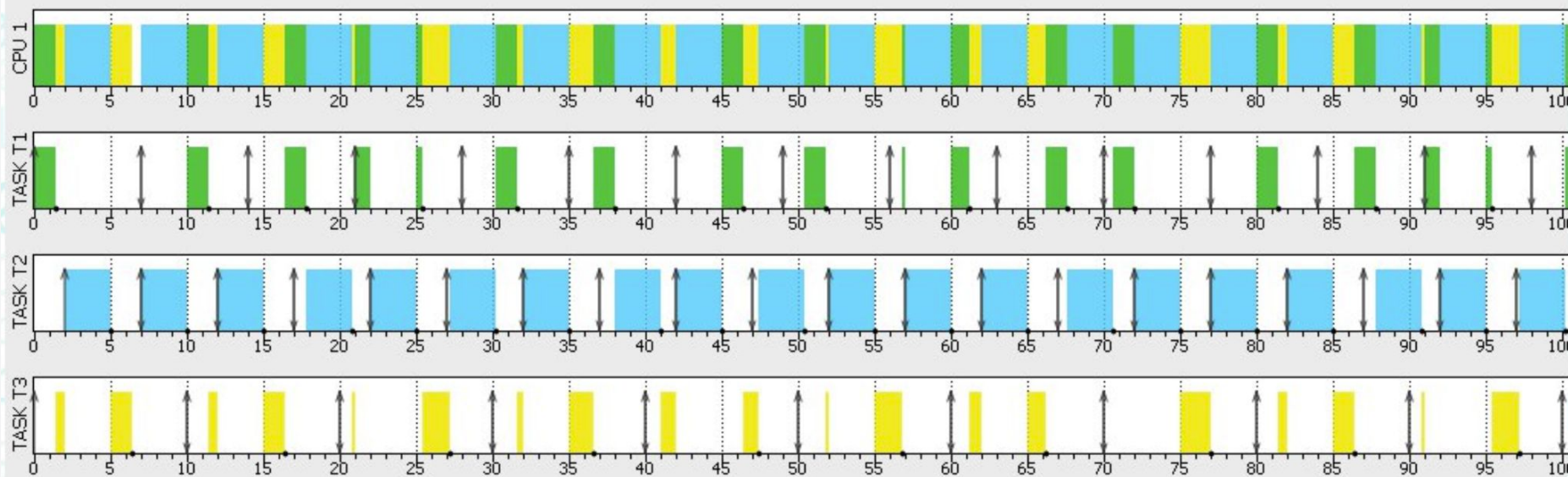
SCHED_RR

Preemptive Round-Robin policy with fixed time slices for equal-priority tasks.

SCHED_DEADLINE

Hard real-time policy based on Earliest Deadline First (EDF) using Runtime, Period, and Deadline.

Multi-Tasking Timeline and Core CPU Allocation



Linux Threading Architecture: NPTL & pthreads

Multithreading Architecture Models in Linux

1:1 THREADING MODEL

NPTL Implementation

Native POSIX Thread Library: Linux maps every user thread created via `pthread_create()` directly to a kernel-level task (`task_struct` / LWP).

These individual units are managed directly by the standard Linux kernel scheduler across multiple CPU cores.

Key Advantage: Blocking system calls (such as I/O operations) only suspend the calling thread (`TASK_INTERRUPTIBLE`), allowing other active threads within the same process to continue concurrent execution.

CREATION MECHANISMS

Thread Creation

High-Level API: The high-level user space interface is provided via `pthread_create()`, which is a standardized part of the libpthread library inside glibc.

Low-Level System Call: Under the hood, this interface translates directly into the low-level Linux system call: `clone()`.

Shared Resources: To establish a thread instead of a separate process, the system call is executed with specific resource sharing flags:
`CLONE_VM`, `CLONE_FS`, `CLONE_FILES`,
`CLONE_SIGHAND`

Thread Synchronization: Mutexes & Semaphores

Concurrency & Synchronization in Linux

SYNCHRONIZATION MECHANISMS

Race Condition & Critical Region

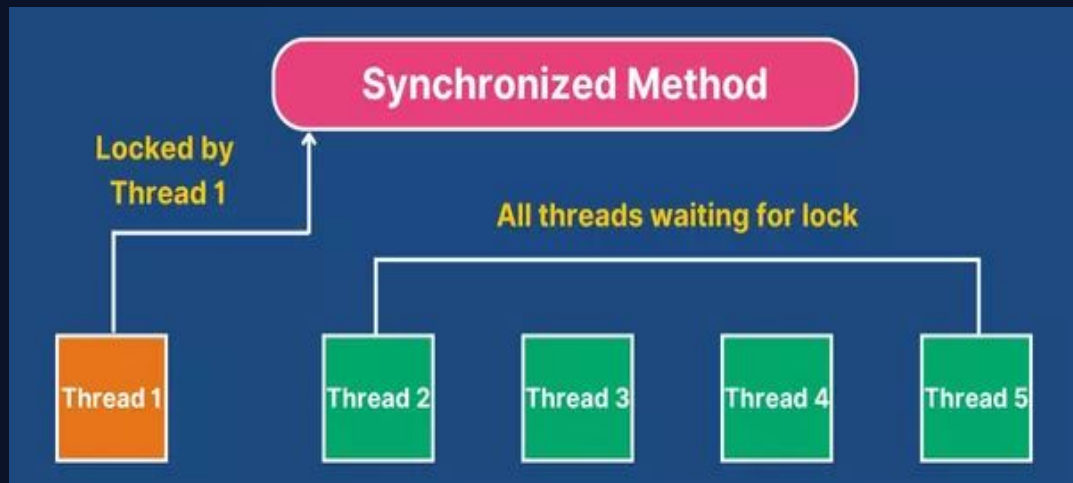
Multiple threads accessing shared memory (global variables) concurrently require synchronization to avoid unpredictable results.

POSIX Mutex (`pthread_mutex_t`)

Mutual exclusion lock (`pthread_mutex_lock` / `pthread_mutex_unlock`). Strictly binary; only the thread that locked the mutex can unlock it.

POSIX Semaphore (`sem_t`)

Atomic integer counter created via `sem_init()` for resource tracking or thread signaling. Decremented via `sem_wait()` (blocks if 0) and incremented via `sem_post()`. Can be shared between threads (`pshared=0`) or distinct Linux processes (`pshared=1`).



Execution Hazards & Conclusions

Synchronization Pitfalls & Conclusions

Deadlock & Priority Inversion

Deadlock in Linux: Circular waiting chain where threads block forever attempting to lock shared resources.

Priority Inversion: High-priority real-time task blocked waiting for a resource held by a lower-priority task.

POSIX Solution: Handled in POSIX via Priority Inheritance protocols (`PTHREAD_PRIO_INHERIT`) to temporarily elevate lower-priority task priority.

Conclusions

Unified Abstraction: Linux unifies processes and threads under a single abstraction (`task_struct`), scheduled via modular policies.

Multi-Core Efficiency: Efficient multithreading with `pthread`s maximizes multi-core processing power when synchronized correctly.

Key Takeaway: Mitigating hazards like deadlocks and priority inversion is essential for robust, deterministic system performance in real-world Linux environments.

Bibliography

Curs Sisteme de operare 2

Universitatea "Politehnica" Timisoara

Departamentul Calculatoare

Digital Signal Processing Labs Timisoara

Valentin STÂNGACIU

Ver. 1.83/2025 Copyright © Valentin Stângaciu